

Faculty of Science and Engineering

Department of Advanced Computing Sciences

Building and Mining Knowledge Graphs

Project Report

MSc AI & MSc DSDM

Author

Federico Antolini (i6457170)

Modeling a Knowledge Graph for Vulnerability and Real-World Exploitation Analysis

March 13, 2026

1 Abstract

This project develops a knowledge graph to integrate vulnerability data from multiple cybersecurity datasets, including the National Vulnerability Database (CVE), the MITRE Common Weakness Enumeration (CWE), and the CISA Known Exploited Vulnerabilities (KEV) catalog. These sources contain complementary information about vulnerabilities, software weaknesses, and real-world exploitation but are difficult to analyze jointly due to heterogeneous formats. The proposed approach constructs an RDF knowledge graph linking vulnerabilities, weaknesses, affected software products, and exploitation records. The graph is validated using SHACL constraints to ensure structural consistency. SPARQL queries are then used to analyze relationships within the integrated data. The results highlight patterns such as software products most frequently affected by exploited vulnerabilities and inconsistencies between weakness annotations across datasets. This work demonstrates the usefulness of knowledge graph technologies for integrating heterogeneous cybersecurity datasets and enabling reproducible, graph-based analysis of vulnerability ecosystems

2 Introduction

Every year thousands of software vulnerabilities are disclosed in public databases, affecting a wide range of software systems and exposing organizations to cyber attacks.

Several datasets provide structured information about these vulnerabilities. The National Vulnerability Database (NVD) documents vulnerabilities through the Common Vulnerabilities and Exposures (CVE) system and includes metadata such as severity scores and descriptions. The Common Weakness Enumeration (CWE) provides a hierarchical taxonomy of the underlying programming weaknesses, while the CISA Known Exploited Vulnerabilities (KEV) catalog identifies vulnerabilities that have been confirmed to be exploited in real-world attacks.

However, these datasets follow different structures and are typically distributed as independent data sources. As a result, cross-dataset analysis requires substantial preprocessing and makes it difficult to explore relationships between vulnerabilities, weaknesses, software products, and exploitation events.

This project addresses these limitations by constructing a knowledge graph that integrates NVD, CWE, and KEV data, linking vulnerability records, weakness classifications, and exploitation indicators to enable structured analysis through graph-based queries.

To achieve this goal, the project pursues the following objectives:

- Design a conceptual model representing vulnerabilities, software products, organizations, and weakness hierarchies.
- Integrate heterogeneous cybersecurity datasets into a unified RDF knowledge graph using established vocabularies.

- Ensure data quality and structural consistency through SHACL validation.
- Enable analytical exploration through SPARQL queries.

3 Related Work

Research on software vulnerabilities spans multiple research directions including vulnerability discovery, attack modeling, and empirical studies of exploitation dynamics.

A first stream of research investigates the economics of vulnerability discovery and disclosure. Rescorla E. (Is finding security holes a good idea? IEEE Security & Privacy, 2005) analyzes trends in vulnerability discovery and discusses whether identifying software security flaws contributes to improved security outcomes. Arora A., Telang R., and Xu H. (Optimal policy for software vulnerability disclosure, 2010) examine the policy implications of vulnerability disclosure mechanisms and how incentives influence vulnerability reporting and patching.

Another line of research focuses on graph-based representations of cyber attacks. Noel S. and Jajodia S. (Managing attack graph complexity through visual hierarchical aggregation, 2009) propose techniques for modeling attack paths through hierarchical attack graphs.

More recent work examines the relationship between vulnerability characteristics and real-world exploitation. Allodi L. and Massacci F. (Comparing Vulnerability Severity and Exploits Using Case-Control Studies, 2014) show that commonly used severity metrics do not always correspond to the actual likelihood of exploitation.

Recent research has also explored the use of knowledge graphs in cybersecurity, proposing approaches to integrate heterogeneous cybersecurity information such as vulnerabilities, attack patterns, and threat intelligence.

Despite these advances, most studies analyze vulnerability datasets using statistical approaches or graph models designed for specific analytical tasks. Comparatively less attention has been devoted to the semantic integration of heterogeneous cybersecurity datasets while preserving the hierarchical structure of software weaknesses.

This project contributes to this direction by constructing a knowledge graph that integrates vulnerability records, weakness taxonomies, and exploitation indicators within a unified semantic framework, enabling structured cross-dataset queries, reproducible analysis, and traceability of the original data sources.

4 Methodology

The project follows a knowledge graph construction pipeline consisting of five main stages: data collection, ontology design, data transformation, graph validation, and analytical querying.

4.1 Data Sources

The knowledge graph integrates information from three publicly available cybersecurity datasets.

The National Vulnerability Database (NVD) provides detailed records of publicly disclosed vulnerabilities through the CVE system. These records include textual descriptions, vulnerability severity scores based on the CVSS standard, and references to associated software weaknesses.

The Common Weakness Enumeration (CWE) dataset provides a hierarchical taxonomy of software weaknesses representing the underlying causes of vulnerabilities. This taxonomy enables grouping vulnerabilities according to broader weakness categories.

The CISA Known Exploited Vulnerabilities (KEV) catalog identifies vulnerabilities that have been confirmed to be exploited in real-world attacks. Integrating KEV information makes it possible to distinguish between disclosed vulnerabilities and those actively exploited in the wild.

For feasibility reasons, the NVD dataset used in this project was restricted to vulnerabilities published in the year 2025.

4.2 Ontology Design

A conceptual schema was designed to represent the main entities and relationships across the integrated datasets. The ontology includes classes representing vulnerabilities, software weaknesses, software products, organizations, and exploited vulnerability records. In addition to the project-specific vocabulary, established semantic web vocabularies such as SKOS and PROV-O were used to represent weakness hierarchies and provenance information.

The main classes in the model are `Vulnerability`, `Weakness`, `KEVRecord`, `SoftwareApplication`, and `Organization`.

Object properties capture relationships between these entities, such as vulnerabilities affecting software products, vulnerabilities associated with specific weakness categories, and KEV records referencing exploited vulnerabilities. Datatype properties represent attributes such as vulnerability severity scores, textual descriptions, and dates.

The conceptual structure of the ontology is illustrated in Appendix A. In the diagram, green nodes represent classes, green edges represent object properties between classes, and blue nodes represent the ranges of datatype properties, which are represented as black edges.

4.3 Data Integration and Graph Construction

The datasets were first processed in CSV format and then transformed into RDF triples according to the conceptual schema defined for the project. The implementation was carried out in Python using the RDFLib library for RDF graph construction and management.

To improve data traceability and support FAIR data principles, provenance metadata were also modeled using the W3C PROV-O ontology. This allows entities in the knowledge graph to be linked to their original data sources and documents the knowledge graph construction process.

During this stage, CVE records from the NVD dataset were linked to their associated CWE weakness categories. Software products and vendors were extracted from configuration information and represented as instances of `SoftwareApplication` and `Organization`. KEV entries were then connected to the corresponding vulnerabilities in order to represent confirmed exploitation events.

The resulting RDF graph integrates vulnerability records, weakness hierarchies, software products, and exploitation indicators within a single semantic structure.

4.4 SHACL Validation

To ensure structural consistency and data quality, SHACL constraints were defined for the main classes and properties in the ontology. These constraints specify datatype restrictions, cardinality constraints, and class relationships for key properties such as vulnerability identifiers, severity scores, and software relationships.

Validation was performed using the `pySHACL` library. During this process several inconsistencies were identified, including missing identifiers and datatype mismatches. Records that violated structural constraints were removed or corrected, resulting in a cleaned and validated knowledge graph used for the subsequent analysis.

4.5 SPARQL-Based Analysis

After validation, the knowledge graph was explored using SPARQL queries in order to analyze relationships between vulnerabilities, weakness categories, software products, and exploited vulnerabilities.

5 Results

5.1 Knowledge Graph Construction

The final knowledge graph integrates NVD vulnerability data, the CWE taxonomy, and the CISA Known Exploited Vulnerabilities (KEV) catalog for the year 2025.

SHACL validation identified several structural inconsistencies, leading to the removal of some invalid entries and the creation of a cleaned knowledge graph used for the analysis.

After validation and cleaning, the resulting knowledge graph contains the entities summarized in Table 1.

Entity Type	Count
Vulnerabilities (CVE)	43200
Weaknesses (CWE)	944
Software Application (Products)	8670
Organizations (Vendors)	3096
KEV Records	165
Total RDF Triples	647627

Table 1: Size of the resulting knowledge graph.

5.2 Analytical Results

After constructing and validating the knowledge graph, SPARQL queries were used to analyze three aspects of the integrated data: the relationship between weakness families and vulnerability severity, differences in weakness classifications across datasets, and the distribution of exploited vulnerabilities across vendors and software products.

The SPARQL queries used in the analysis are included in the section titled "SPARQL Queries" of the *FixedSoftwareVulnerabilitiesGraph.ipynb* notebook provided in the project zip folder, allowing the results to be reproduced. In the tables, weakness categories are reported using their CWE identifiers rather than full RDF URIs for readability.

5.2.1 Weakness Families and Vulnerability Severity

The first analysis investigates the relationship between software weakness families and vulnerability severity. Using the hierarchical structure of the CWE taxonomy, vulnerabilities disclosed in 2025 were grouped according to the broader weakness family associated with their underlying CWE category.

For each weakness family, the average CVSS base score of the corresponding vulnerabilities was computed. To ensure statistical relevance, the analysis only considers weakness families associated with at least ten vulnerabilities.

Table 2 presents the weakness families with the highest average vulnerability severity.

The results indicate that some weakness families are systematically associated with higher severity vulnerabilities. This suggests that certain classes of programming errors may lead to more critical security risks. By leveraging the hierarchical structure of the CWE taxonomy, the knowledge graph enables aggregation of vulnerabilities at the family level.

Parent CWE ID (Parent Label)	Number of Vulnerabilities	Avg CVSS Score
CWE-602 (Client-Side Enforcement of Server-Side Security)	11	8.3
CWE-190 (Integer Overflow or Wraparound)	11	8.273
CWE-306 (Missing Authentication for Critical Function)	156	8.226

Table 2: Weakness families with the highest average vulnerability severity.

5.2.2 Differences in Weakness Classification Between KEV and NVD

The second analysis investigates differences in weakness classification between the NVD vulnerability dataset and the CISA Known Exploited Vulnerabilities (KEV) catalog: vulnerabilities appearing in the KEV dataset were compared with their corresponding records in the NVD dataset. The query identifies cases where the KEV catalog associates a vulnerability with a weakness category that does not appear among the weaknesses listed in the corresponding NVD record. The query returned six vulnerabilities for which the weakness classification differs between the KEV and NVD datasets. Table 3 reports selected examples of these cases.

CVE ID	NVD Weakness ID (NVD Label)	KEV Weakness ID (KEV Label)
CVE-2025-27920	CWE-24 (Path Traversal: '../filedir')	CWE-22 (Improper Limitation of a Pathname...)
CVE-2025-32706	CWE-20 (Improper Input Validation)	CWE-122 (Heap-based Buffer Overflow)
CVE-2025-32756	CWE-121 (Stack-based Buffer Overflow)	CWE-124 (Buffer Underwrite)

Table 3: Examples of vulnerabilities where KEV and NVD report different weakness categories.

These differences highlight potential inconsistencies or complementary information across vulnerability datasets. In particular, exploited vulnerabilities may involve multiple underlying weaknesses, and different datasets may emphasize different aspects of the vulnerability. Identifying these discrepancies is therefore useful both for assessing the consistency of vulnerability classification and for revealing additional weakness patterns that may emerge during real-world exploitation.

5.2.3 Products and Vendors Most Affected by Exploited Vulnerabilities

The final analysis examines the distribution of exploited vulnerabilities across software products and their corresponding vendors.

By linking vulnerability records to affected software applications and identifying those included in the KEV catalog, it is possible to determine which vendor–product combinations are most frequently associated with real-world exploitation events.

Table 4 reports the vendor and software product with the highest number of exploited vulnerabilities in the dataset.

Vendor	Software Product	Exploited Vulnerabilities
Microsoft	Windows Server 2022 23H2	25

Table 4: Vendor and software product most frequently associated with exploited vulnerabilities in the 2025 dataset.

This analysis shows that exploitation events tend to concentrate around specific vendor–product ecosystems, highlighting the importance of prioritizing security efforts for widely deployed software systems.

6 Discussion

The results demonstrate the usefulness of semantic data integration for exploring structural patterns in vulnerability datasets. By linking vulnerability records, weakness taxonomies, and exploitation indicators in a unified RDF framework, the project enables analytical queries that are difficult to perform using the original datasets independently.

The analysis of weakness categories and vulnerability severity suggests that certain classes of software weaknesses tend to be associated with higher CVSS scores. This highlights the importance of understanding the structural causes of vulnerabilities rather than focusing solely on individual vulnerability instances.

The comparison between NVD vulnerability records and the KEV catalog also revealed differences in weakness classification. In some cases, vulnerabilities associated with a particular weakness in the NVD dataset appear linked to different weakness categories in the KEV dataset. These discrepancies may reflect differences in classification methodologies or updates in vulnerability analysis over time.

The analysis of exploited vulnerabilities across software products further shows that exploitation events tend to concentrate around a limited number of software systems. This observation suggests that widely deployed software may present larger attack surfaces and become attractive targets for attackers.

Despite these results, several limitations should be acknowledged. First, the dataset used in this project only includes vulnerabilities disclosed in 2025. While this restriction allowed the project to remain computationally manageable, it limits the possibility of identifying long-term trends in vulnerability discovery, weakness distribution, and exploitation patterns.

Second, the integration process required substantial data preprocessing and validation. During graph construction, several structural inconsistencies were identified through SHACL validation, including datatype mismatches and missing mandatory properties. These issues illustrate the challenges involved in integrating heterogeneous cybersecurity datasets.

Future work could extend the dataset beyond the single year considered in this project. Incorporating multiple years of NVD and KEV data would enable the study of temporal dynamics in vulnerability exploitation and provide a richer dataset for analysis. Another promising direction is the integration of machine learning techniques with the knowledge graph. In particular, Graph Neural Networks (GNNs) could leverage the relational structure of the graph to identify patterns associated with exploited vulnerabilities. Combined with retrieval-augmented generation (RAG) techniques, such approaches could support predictive analysis of vulnerability risk patterns.

7 Conclusions

This project presented a knowledge graph for integrating vulnerability data and exploitation information. By transforming heterogeneous cybersecurity datasets into a unified RDF representation, the project enabled structured exploration of relationships between vulnerabilities, software weaknesses, affected products, and real-world exploitation events.

The resulting knowledge graph supports analytical queries that reveal patterns in vulnerability severity, weakness classifications, and the distribution of exploited vulnerabilities across software products.

The project also demonstrated the usefulness of semantic technologies for integrating and validating cybersecurity datasets. SHACL constraints helped identify structural inconsistencies during graph construction and improved overall data quality.

Overall, the results show that knowledge graph approaches provide a flexible framework for cybersecurity data analysis. Future work could extend the dataset to multiple years of vulnerability records and explore graph-based machine learning techniques, such as Graph Neural Networks, to support predictive analysis of vulnerability exploitation risks.

References

W3C. Resource Description Framework (RDF) 1.1 Concepts and Abstract Syntax.

W3C. Shapes Constraint Language (SHACL).

W3C. SPARQL 1.1 Query Language.

NIST. (2024). National Vulnerability Database (NVD). National Institute of Standards and Technology.

ENISA. (2023). ENISA Threat Landscape 2023. European Union Agency for Cybersecurity.

MITRE. (2023). Common Weakness Enumeration (CWE) Overview.

Rescorla, E. (2005). Is finding security holes a good idea? IEEE Security & Privacy.

Arora, A., Telang, R., Xu, H. (2010). Optimal policy for software vulnerability disclosure.

Noel, S., Jajodia, S. (2009). Managing attack graph complexity through visual hierarchical aggregation.

Allodi, L., Massacci, F. (2014). Comparing Vulnerability Severity and Exploits Using Case-Control Studies.

A Knowledge Graph Conceptual Schema

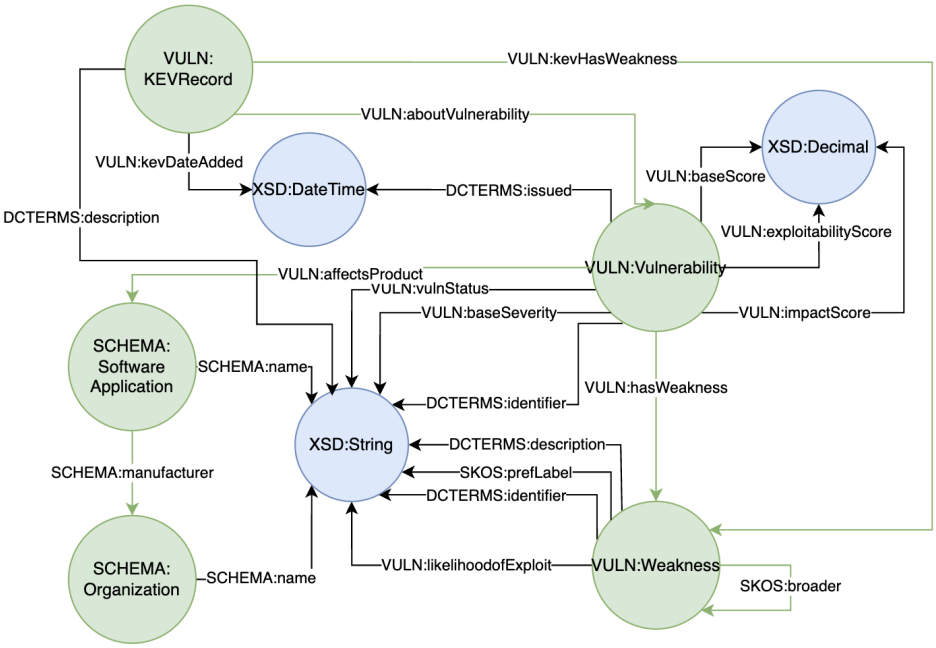


Figure 1: Conceptual schema of the vulnerability knowledge graph.